Assignment Report

SFWRENG 3S03 (2026W)

Assignment 1

Author(s)

Om Patel	patelo11@mcmaster.ca	PATELO11
Soutrik Guhathakurta	guhathas@mcmaster.ca	GUHATHAS
Jim Software	js@mcmaster.ca	MACID789
Joan Engineer	je@mcmaster.ca	MACIDABC

GitHub URL

`https://github.com/jondoe/a1repo`

February 20, 2026

Original template credits: Samuel Zurowski and Charles Barone. Adapted for McMaster SFWRENG by Dr. Istvan David.

1 Task 1

Dr. David ▶*Elaborate on each task in a separate section. Delete these comments in your final report.*◀

2 Task 2

3 Task 3: Testing parts of large systems

3.1 (a)

The agent's behaviour can be decoupled using **Dependency Injection** and **Test Doubles** (specifically, stubs/fakes) behind the provided `CatanAgent` interface.

The core engine should solely rely on:

- game rules/state transitions, and
- the contract `CatanAgent` (inputs/outputs)

This allows testing the core module with deterministic, pre-scripted agent choices, so tests focus on legality checks and rule execution rather than RL quality. It also makes later replacement easy: random, heuristic, RL, or human-network agents can all plug in without changing core logic.

3.2 (b)

A minimal implementation of the design is implemented in `src/catan`:

- `CatanAgent.java`: given interface (core dependency boundary).
- `AgentPolicy.java`: behavior strategy abstraction used for composition.
- `PolicyBackedCatanAgent.java`: adapter that implements `CatanAgent` by delegating to an injected policy.
- `ScriptedPolicy.java`: deterministic test double with queued moves.
- `CatanCoreEngine.java`: core loop; validates move legality and applies a safe fallback (`PASS`) if illegal.
- `GameState.java`, `Move.java`, enums: minimal domain model.
- `Demo.java`: runs four agents to show separation in practice.

Key point: the engine never knows whether a move came from a stub, heuristic, or RL policy. That behavior is injected.

3.3 (c)

A common solution from a developer unfamiliar with these techniques is to place decision logic directly inside the engine, e.g., `if(agentType == RL) ... else ...`, or to hardcode random/heuristic behavior in core classes.

Problems with that approach and benefits of mine:

- **High coupling**: rule code and behavior code become tangled. In the current design, the core only sees `CatanAgent`.
- **Poor testability**: non-deterministic behavior makes core tests flaky. Scripted policies make tests reproducible.
- **Open/Closed violation**: every new agent type forces edits to core logic. With injection, new policies are added without touching engine code.
- **Higher maintenance risk**: bugs in learning code can destabilize rule validation. Separation keeps failures localized and easier to debug.

3.4 (d)

I would communicate to:

- technical peers (design session + code walkthrough),
- tech lead/manager (delivery and risk argument), and
- QA/testing members (how deterministic doubles improve test quality).

How and what I would communicate:

- short architecture note (problem, decision, tradeoffs, examples),
- live demo with the implemented files showing swappable policies,
- test strategy showing deterministic agent stubs for core-rule tests,
- team conventions: "core depends on interfaces, behavior injected".

Expectations:

- shared vocabulary (DI, test double, strategy),
- more stable and faster tests,
- easier onboarding and safer extension to RL agents.

With respect to the CEAB engineering practice: this reflects disciplined design, verification mindset, communication, professionalism, and continuous process improvement. It applies software engineering knowledge to reduce risk while improving quality and maintainability.

4 Task 4: Test-Driven Development(TDD)

4.1 (a)

The following small initial unit test suite for `divide()` can be used to verify basic functionality and sign semantics:

- `testDividePositiveNumbers`: verifies normal behaviour (`5.0 / 2.0 = 2.5`)
- `testDivideNegativeByPositive`: verifies sign semantics (`-9.0 / 3.0 = -3.0`)

These tests are implemented in `src/calculator/CalculatorTddTest.java`. They define the first iteration's target behaviour before implementing full edge-case handling.

4.2 (b)

The basic functionality has been implemented in `src/calculator/Calculator.java` until the tests passed:

- method `divide(double numerator, double denominator)`
- returns `numerator / denominator` for valid normal inputs

The tests were then executed from the command line and were confirmed to pass.

4.3 (c)

After iteration 1, the functionality is not complete. The first tests only validate nominal arithmetic behaviour and one sign case. Missing behaviour becomes obvious when compared with code one would write non-TDD in one pass, where developers often immediately think about exceptional cases.

Important missing behaviours:

- division by zero (`0.0` and `-0.0`)
- invalid floating input such as `NaN`
- explicit and documented error semantics (which exception and why)

TDD made this visible quickly: once minimal tests pass, the next step is to identify what behaviour is still unspecified. If a behaviour is not in tests, it is effectively not part of the contract yet.

4.4 (d)

The test suite was then expanded by adding a second TDD iteration focused on edge cases:

- `testDivideByZeroThrows`: expects `ArithmeticException` for denominator `0.0`
- `testDivideNegativeZeroThrows`: expects `ArithmeticException` for denominator `-0.0`

- `testDivideNaNThrows`: expects `IllegalArgumentException` if an argument is `NaN`

These tests are also in `src/calculator/CalculatorTddTest.java`. They extend the contract from "works on normal numbers" to "fails safely on invalid input."

4.5 (e)

The `divide()` method was then updated to satisfy the expanded suite:

- validate inputs and reject `NaN` with `IllegalArgumentException`
- reject zero denominator (including `-0.0`) with `ArithmeticException`
- keep normal arithmetic behaviour unchanged

This final behaviour is implemented in `src/calculator/Calculator.java`.

4.6 (f)

I would use TDD in contexts where correctness and regression safety are important: utility/-domain logic, business rules, parsers, and modules with clear input/output contracts.

I would use it less strictly for heavy UI prototyping or exploratory tasks where requirements change rapidly by design.

Element I liked most:

- it forces explicit behaviour contracts and prevents overbuilding

Hardest element to manage:

- selecting the right test granularity and not overfitting implementation details while still covering edge cases

Overall, this exercise showed the TDD cycle clearly across two iterations: start with minimal behaviour, then expand the contract with edge cases and update implementation accordingly.